创新创业实践

Project3

说明文档

郎泓森

202200460010

一、代码解释

使用 Poseidon2 哈希算法验证两个私有输入的哈希值等于公开声明的哈希值，实现了高效的可验证计算协议，允许用户在隐藏私有输入（privateInp）的前提下，向验证者证明他们知道生成指定哈希（publicHash）的原始数据。

1. SBox（非线性变换）

```
template SBox() {
  signal input  inp;
  signal output out;
  signal x2 <== inp*inp;
  signal x4 <== x2*x2;
  out <== inp*x4; // out = inp^5
}
```

执行非线性变换：计算输入的五次幂（inp^5），增强密码学的混淆性

2. InternalRound（内部轮函数）

```
template InternalRound(i) {
  // 只对第一个元素加常数+SBox
  component sb = SBox();
  sb.inp <== inp[0] + round_consts[i];

  // 线性混合层
  out[0] <== 2*sb.out + inp[1] + inp[2];
  out[1] <==   sb.out + 2*inp[1] + inp[2];
  out[2] <==   sb.out + inp[1] + 3*inp[2];
}
```

每轮使用 56 个预置常数中的 1 个。仅处理状态数组的第一个元素（节省计算量），使用特定系数矩阵进行线性混合。

3. ExternalRound（外部轮函数）

```
template ExternalRound(i) {
  component sb[3]; // 三个独立SBox
  for(var j=0; j<3; j++) {
    sb[j] = SBox();
    sb[j].inp <== inp[j] + round_consts[i][j];
  }

  // 不同系数的线性混合
  out[0] <== 2*sb[0].out + sb[1].out + sb[2].out;
  out[1] <==   sb[0].out + 2*sb[1].out + sb[2].out;
  out[2] <==   sb[0].out + sb[1].out + 2*sb[2].out;
}
```

使用每轮 3 个独立的预置常数，处理状态数组的所有三个元素，采用全状态非线性变换。

4. LinearLayer（初始线性层）

```
template LinearLayer() {
  out[0] <== 2*inp[0] + inp[1] + inp[2];
  out[1] <==   inp[0] + 2*inp[1] + inp[2];
  out[2] <==   inp[0] + inp[1] + 2*inp[2];
}
```

在置换开始前进行数据扩散，使用特殊的双对角线矩阵增强非线性特性

5. Permutation（核心置换）

```
template Permutation() {
  // 轮函数顺序: 4外部轮 → 56内部轮 → 4外部轮
  component ext[8];  // 8个外部轮
  component int[56]; // 56个内部轮

  // 初始线性层
  component ll = LinearLayer();

  // 64轮处理流程
  ll → 4外部轮 → 56内部轮 → 4外部轮 → 输出
}
```

总轮数：1(LinearLayer) + 4(外部轮) + 56(内部轮) + 4(外部轮) = 65 步，通过外部轮/内部轮的交替设计平衡安全性和效率

6. Poseidon2_2_1（哈希接口）

```
template Poseidon2_2_1() {
  component perm = Permutation();
  perm.inp[0] <== inp[0]; // 输入1
  perm.inp[1] <== inp[1]; // 输入2
  perm.inp[2] <== 0;      // 固定填充
  out <== perm.out[0];    // 取第一个状态作为哈希
}
```

专为 2 个输入设计的哈希方案，第三状态位固定填充 0（t=3 配置）。输出取置换后的第一个状态值。

7. Main（主验证电路）

```
template Main() {
  signal input privateInp[2]; // 私有输入
  signal input publicHash;    // 公开哈希

  component hasher = Poseidon2_2_1();
  hasher.inp[0] <== privateInp[0];
  hasher.inp[1] <== privateInp[1];

  hasher.out === publicHash; // 约束：计算值=公开值
}
```

私有输入的哈希必须等于公开声明的哈希值，通过零知识证明实现：验证者知晓私有输入但无需泄露。

二、算法数学推导

S 盒的数学表示

S 盒对输入inp进行五次幂运算：$S(x) = x^5$

计算$x2 = x * x$

计算$x4 = x2 * x2$

输出$out = x * x4 = x\text{^}5$

S 盒负责提供非线性变换，增强算法的抗密码分析能力

线性层进行线性变换

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

通过线性组合扩散输入差异，实现状态混淆

轮函数的结构

内部轮对第一个元素施加 S 盒并加轮常数

$$sb.inp = x_0 + round\_consts[i]$$

外部轮对每个元素独立施加 S 盒并加轮常数

$$sb[j].inp = x_j + round\_consts[i][j] (j = 0,1,2)$$

置换函数的完整流程

置换由 64 轮操作组成，分层结构如下

初始线性层

$$a^0 = M \cdot x$$

4 轮外部轮

$$a^{k+1} = ExternalRound_k(a^k)(k = 0, \dots, 3)$$

56 轮外部轮

$$a^{k+5} = InternalRound_k(a^{k+4})(k = 0, \dots, 5)$$

4 轮外部轮

$$a^{k+61} = ExternalRound_{k+4}(a^{k+60})(k = 0, \dots, 3)$$

输出

$$y = a^{64}$$

三、Poseidon2 软件实现及 Groth16 证明

编译电路

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ circom poseidon2.circom --r1cs --wasm
template instances: 69
non-linear constraints: 240
linear constraints: 275
public inputs: 0
private inputs: 3
public outputs: 0
wires: 518
labels: 918
Written successfully: ./poseidon2.r1cs
Written successfully: ./poseidon2_js/poseidon2.wasm
Everything went okay
```

输出指令，可以看到编译成功。

可信设置

通过 snarkjs 生成 Powers of tau

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ snarkjs powersoftau new bn128 14 pot14_0000.ptau -v
[DEBUG] snarkJS: Calculating First Challenge Hash
[DEBUG] snarkJS: Calculate Initial Hash: tauG1
[DEBUG] snarkJS: Calculate Initial Hash: tauG2
[DEBUG] snarkJS: Calculate Initial Hash: alphaTauG1
[DEBUG] snarkJS: Calculate Initial Hash: betaTauG1
[DEBUG] snarkJS: Blank Contribution Hash:
                786a02f7 42015903 c6c6fd85 2552d272
                912f4740 e1584761 8a86e217 f71f5419
                d25e1031 afee5853 13896444 934eb04b
                903a685b 1448b755 d56f701a fe9be2ce
[INFO]  snarkJS: First Contribution Hash:
                bc0bde79 80381fa6 42b20975 91dd83f1
                ed15b003 e15c3552 0af32c95 eb519149
                2a6f3175 215635cf c10e6098 e2c612d0
                ca84f1a9 f90b5333 560c8af5 9b9209f4
```

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ snarkjs powersoftau contribute pot14_0000.ptau pot14_0001.ptau --n
ame="First Contribution" -v
Enter a random text. (Entropy): awfffk
[DEBUG] snarkJS: Calculating First Challenge Hash
[DEBUG] snarkJS: Calculate Initial Hash: tauG1
[DEBUG] snarkJS: Calculate Initial Hash: tauG2
[DEBUG] snarkJS: Calculate Initial Hash: alphaTauG1
[DEBUG] snarkJS: Calculate Initial Hash: betaTauG1
[DEBUG] snarkJS: processing: tauG1: 0/32767
[DEBUG] snarkJS: processing: tauG1: 16384/32767
[DEBUG] snarkJS: processing: tauG2: 0/16384
[DEBUG] snarkJS: processing: tauG2: 8192/16384
[DEBUG] snarkJS: processing: alphaTauG1: 0/16384
[DEBUG] snarkJS: processing: betaTauG1: 0/16384
[DEBUG] snarkJS: processing: betaTauG2: 0/1
[INFO]  snarkJS: Contribution Response Hash imported:
                 96c83cfe 47622ba9 f9ce5b78 9f363d0d
                 adb00b5c e89be25b 986c37a9 e44da666
                 17fd2fbf fd95f193 e45de01c bcf099de
                 4bb4c145 82d0538c 4e12f876 26d7b182
[INFO]  snarkJS: Next Challenge Hash:
                 e653cfff da923dd6 87fdf9d5 dcbedb4f
                 8e3127b8 f0d57728 66edbf0d f53ee8fc
                 f1ee66f9 b8615c13 ebf34e57 2b5d2ea1
                 064a41ea 407a8acd 1d1cd300 b4516c4b
```

```
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  1/2 1/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  1/2 5/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  2/2 3/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  1/2 3/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  2/2 0/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  2/2 2/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  2/2 6/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  2/2 4/8
[DEBUG] snarkJS: betaTauG1: fft 14 join  13/14  1/2 4/8
[DEBUG] snarkJS: betaTauG1: fft  14  join: 14/14
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 10/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 6/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 1/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 11/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 2/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 5/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 4/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 13/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 3/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 8/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 14/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 15/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 12/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 7/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 0/16
[DEBUG] snarkJS: betaTauG1: fft 14 join  14/14  1/1 9/16
```

Groth16 证明

生成初始 key

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ snarkjs groth16 setup poseidon2.r1cs pot14_final.ptau poseidon2_00
00.zkey
[INFO]  snarkJS: Reading r1cs
[INFO]  snarkJS: Reading tauG1
[INFO]  snarkJS: Reading tauG2
[INFO]  snarkJS: Reading alphatauG1
[INFO]  snarkJS: Reading betatauG1
[INFO]  snarkJS: Circuit hash:
                 766467c4 98320a12 38e12e66 5293bbff
                 fd1c752a 343f1bc2 c5b56cb9 41ce9e67
                 0cb2150c 533751c6 910dcf41 6d91633d
                 44a41b30 2b65e18d 217f0a61 5171e16c
```

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ snarkjs zkey contribute poseidon2_0000.zkey poseidon2_0001.zkey --
name="Contributor" -v
Enter a random text. (Entropy): alflhwf
[DEBUG] snarkJS: Applying key: L Section: 0/517
[DEBUG] snarkJS: Applying key: H Section: 0/1024
[INFO]  snarkJS: Circuit Hash:
                 766467c4 98320a12 38e12e66 5293bbff
                 fd1c752a 343f1bc2 c5b56cb9 41ce9e67
                 0cb2150c 533751c6 910dcf41 6d91633d
                 44a41b30 2b65e18d 217f0a61 5171e16c
[INFO]  snarkJS: Contribution Hash:
                 82e2f14f 2e0b6a94 e74f3be9 28db8eb8
                 b11ceb0c 539d751f 3db8c3d8 862cd382
                 10a6bc16 36f47d11 d35af98c 9942bc79
                 e90fb76f 17b0c628 9518bea1 e4f03fb0
```

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ snarkjs zkey export verificationkey poseidon2_0001.zkey verificati
on_key.json
[INFO]  snarkJS: EXPORT VERIFICATION KEY STARTED
[INFO]  snarkJS: > Detected protocol: groth16
[INFO]  snarkJS: EXPORT VERIFICATION KEY FINISHED
```

计算见证

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ cd poseidon2_js
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3/poseidon2_js$ node generate_witness.js poseidon2.wasm ../input.json
 ../witness.wtns
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3/poseidon2_js$ cd ..
```

生成可信证明

```
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ snarkjs groth16 prove poseidon2_0001.zkey witness.wtns proof.json
public.json
(base) wolfored@LAPTOP-BJ66GTQI:/mnt/e/github/IAE_practice/Project3$ snarkjs groth16 verify verification_key.json public.json proof.jso
n
[INFO]  snarkJS: OK!
```

生成及验证证明成功。